

TEST PLAN TEMPLATE

Generic · Reusable · Production-Ready

Project Name	[Project / Product Name]
Version	[v1.0]
Environment	[Staging / QA / Production]
Prepared By	[Name], [Role]
Reviewed By	[Name], [Role]
Date	[DD Month YYYY]
Status	[Draft / In Review / Final]

Instructions: Replace all bracketed fields [] with project-specific information before distribution.

1. Test Plan Overview

1.1 Objective

This test plan defines the strategy, scope, approach, and schedule for testing [Project / Feature Name]. The goal is to validate that the product meets defined requirements, is free of critical defects, and delivers a reliable, consistent user experience before release.

1.2 Scope

In Scope

- All user-facing features described in the product requirements document (PRD)
- Core user flows: [e.g., registration, login, checkout, search, form submission]
- UI/UX validation against approved design specifications
- Cross-browser and cross-device coverage as defined in Section 4
- API endpoints directly consumed by the frontend (integration layer)
- Regression coverage for all previously verified functionality

Out of Scope

- Third-party services beyond integration smoke tests (e.g., payment gateways, external APIs)
- Backend admin panels or CMS interfaces unless explicitly requested
- Performance load testing (handled separately)
- Security penetration testing (handled by dedicated security team)
- Features not yet deployed to the test environment

2. Test Strategy

2.1 Approach

- Risk-based prioritization: test cases are ordered by criticality. P1 flows are covered before P2/P3.
- Manual testing covers exploratory, UX, and edge-case scenarios not suitable for automation.
- Automated testing covers regression, smoke, and repetitive functional flows using [e.g., Playwright / Cypress / Selenium].
- All test results are tracked in [e.g., Jira / TestRail / Zephyr]. No verbal-only sign-offs accepted.
- Defects are logged with reproduction steps, environment info, and severity classification.

2.2 Test Levels

- **Unit:** Unit Testing
 - Assumed covered by the development team. QA will not duplicate unit-level validation.
- **Integration:** Integration Testing
 - Validates that system components interact correctly — API to UI, form to backend, state across pages.
- **E2E:** End-to-End (E2E) Testing
 - Full user journey scenarios executed from the frontend. Automated with [tooling].
- **Regression:** Regression Testing
 - Executed after each build, hotfix, or sprint. Covers all previously validated P1/P2 scenarios.
- **Exploratory:** Exploratory Testing
 - Unscripted, time-boxed sessions focused on high-risk modules and edge cases.

3. Test Types

3.1 Functional Testing

Verifies that every feature behaves as described in the requirements. Each user-facing action — form submissions, navigation, filters, modals, CTAs — is tested against an expected outcome.

- All input fields validated for correct behavior, error handling, and boundary conditions
- All navigation links verified to route to correct destinations
- All CTAs trigger the correct action or redirect
- All state changes (filters applied, items saved, forms submitted) persist correctly

3.2 Regression Testing

Executed after every bug fix, hotfix, or new feature deployment. Prevents previously working functionality from breaking due to code changes.

- Automated regression suite covers all P1 and P2 test cases
- Suite is triggered on every build deployed to the QA environment
- Failures block the build from progressing to UAT or production

3.3 UI / UX Testing

Validates visual accuracy and user experience consistency against approved designs and brand guidelines.

- Layout, alignment, and spacing match design specifications
- Typography, color palette, and iconography are applied consistently
- Responsive behavior verified across target screen widths
- Loading states, empty states, and error states are all handled gracefully

3.4 Cross-Browser / Cross-Device Testing

Ensures consistent behavior and rendering across all supported browsers and device types.

- Covered in Section 4 (Test Environment) — platforms, browsers, and resolutions defined there
- Critical flows are executed on each target combination; full suite on primary browser only

4. Test Environment

4.1 Environments

- Primary test environment: [e.g., <https://staging.yourapp.com>]
- Environment must be stable and on a deployment freeze during test execution
- Production environment is off-limits for testing unless explicitly approved for smoke tests only

4.2 Devices & Platforms

- Desktop: 1440px and 1920px viewport widths (primary)
- Mobile: 375px (iPhone 14) and 390px (iPhone 14 Pro) — if mobile is in scope

- Tablet: 768px (iPad) — if tablet is in scope

Browsers:

- Google Chrome (latest stable)
- Mozilla Firefox (latest stable)
- Microsoft Edge (Chromium-based, latest)
- Safari (macOS, latest) — required if target audience includes Apple users

4.3 Tools

- [Test Management] — e.g., Jira + Zephyr / TestRail / Notion
- [Automation Framework] — e.g., Playwright / Cypress / Selenium
- [Bug Tracking] — e.g., Jira / Linear / GitHub Issues
- [Performance] — e.g., Lighthouse, GTmetrix (if in scope)
- [Screen Recording / Evidence] — e.g., Loom, browser DevTools

4.4 Test Data Requirements

- Dedicated test accounts provisioned before execution begins — no shared or production credentials
- Form submissions use a designated test email domain (e.g., @testqa.yourapp.com)
- Edge case data prepared in advance: empty states, max-length inputs, special characters
- Any database seeds or fixtures required must be confirmed with the dev team before execution

5. Test Schedule / Timeline

Adjust durations to match your actual sprint or release cycle. 'X days' is a placeholder.

Phase	Duration	Activities	Owner
Test Planning	X days	Scope definition, risk analysis, test plan review & sign-off	QA Lead
Test Design	X days	Test case writing, test data preparation, environment validation	QA Engineers
Test Execution	X days	Manual & automated test runs, defect logging, daily status updates	QA Team
Regression Testing	X days	Full regression suite post bug fixes, automated + manual	QA Engineers
UAT Support	X days	Support stakeholders during UAT, log UAT-reported issues	QA Lead
Test Closure	X days	Test summary report, metrics review, retrospective	QA Lead

6. Entry & Exit Criteria

6.1 Entry Criteria

Testing does not begin until all of the following conditions are met:

- Test environment is deployed, accessible, and stable
- All in-scope features are deployed to the test environment
- Test cases have been reviewed and approved by the QA Lead
- Test data and test accounts are provisioned and verified
- Design specifications and acceptance criteria are finalized — no open ambiguities
- Smoke test passes: all critical pages load and core flows are accessible

6.2 Exit Criteria

Testing is considered complete when all of the following conditions are met:

- 100% of P1 (Critical) test cases executed
- Minimum 95% of P2 (High) test cases executed
- Zero open Critical defects; zero open High defects blocking core flows
- All discovered defects are logged with reproduction steps and severity classification
- Regression suite executed with no new failures introduced
- Test summary report reviewed and signed off by QA Lead and Product Manager

7. Risks & Mitigation

The following risks have been identified and assessed for this test cycle:

Risk	Likelihood	Impact	Mitigation
Test environment is unstable or frequently reset	Medium	High	Define environment freeze window before execution begins
Incomplete or changing requirements late in cycle	High	High	Freeze scope before test execution; track changes via CR process
Insufficient test data for edge cases	Medium	Medium	Prepare test data set as part of test prep phase
Key team members unavailable during execution	Low	High	Cross-train team members; document all test scenarios
Defect backlog delays exit criteria sign-off	Medium	High	Triage defects daily; escalate blockers to PM within 24 hours
Automation scripts fail on environment changes	Medium	Medium	Run smoke suite before full regression; maintain script version control

8. Roles & Responsibilities

Role	Owner	Responsibilities
QA Lead	[Name]	Test plan authorship, execution oversight, sign-off, defect triage, stakeholder reporting
QA Engineer(s)	[Name(s)]	Test case writing, manual and automated test execution, defect logging
Developer	[Name(s)]	Bug fixes, build deployments, unit test coverage, environment support
Product Manager	[Name]	Requirements sign-off, scope decisions, priority calls on defects
DevOps / Infra	[Name]	Environment provisioning, CI/CD pipeline, deployment readiness

How to Use This Template

- Replace all bracketed fields [] with project-specific information before sharing externally.
- Remove any section that does not apply to your project — this template is deliberately comprehensive, not prescriptive.
- Scope should be defined collaboratively with the PM and dev lead before this document is finalized.
- Revisit and update the Risks table at the start of each sprint or release cycle — risks change as the project evolves.
- Version-control this document. Each update should increment the version number and include a brief change note.

Template version 1.0 · Generic & Reusable · Not confidential